配置

Sandboxing

了解 Claude Code 的沙箱 bash 工具如何提供文件系统和网络隔离，以实现更安全、更自主的代理执行。

概述

Claude Code 具有原生沙箱功能，为代理执行提供更安全的环境，同时减少了对持续权限提示的需求。Claude Code 不是要求对每个 bash 命令进行权限批准，而是预先创建定义的边界，使 Claude Code 可以以更低的风险更自由地工作。

沙箱 bash 工具使用操作系统级原语来强制执行文件系统和网络隔离。

为什么沙箱很重要

传统的基于权限的安全性需要对 bash 命令进行持续的用户批准。虽然这提供了控制，但可能导致：

- **批准疲劳**：重复点击”批准”可能导致用户对他们批准的内容关注度降低
- **生产力降低**：持续的中断会减慢开发工作流程
- **自主性受限**：当等待批准时，Claude Code 无法高效工作

沙箱通过以下方式解决这些挑战：

1. **定义清晰的边界**：精确指定 Claude Code 可以访问的目录和网络主机
2. **减少权限提示**：沙箱内的安全命令不需要批准
3. **维护安全性**：尝试访问沙箱外的资源会触发立即通知
4. **启用自主性**：Claude Code 可以在定义的限制内更独立地运行

Claude Code Docs

⚠ 有效的沙箱需要同时进行文件系统和网络隔离。没有网络隔离，被破坏的代理可能会泄露敏感文件，如 SSH 密钥。没有文件系统隔离，被破坏的代理可能会后门系统资源以获得网络访问权限。配置沙箱时，重要的是确保配置的设置不会在这些系统中创建绕过。

工作原理

文件系统隔离

沙箱 bash 工具将文件系统访问限制在特定目录：

- **默认写入行为**：对当前工作目录及其子目录的读写访问
- **默认读取行为**：对整个计算机的读取访问，除了某些被拒绝的目录
- **被阻止的访问**：无法在没有明确权限的情况下修改当前工作目录外的文件
- **可配置**：通过设置定义自定义允许和拒绝的路径

网络隔离

网络访问通过在沙箱外运行的代理服务器进行控制：

- **域名限制**：只能访问批准的域名
- **用户确认**：新的域名请求会触发权限提示
- **自定义代理支持**：高级用户可以对出站流量实施自定义规则
- **全面覆盖**：限制适用于所有脚本、程序和由命令生成的子进程

操作系统级强制执行

沙箱 bash 工具利用操作系统安全原语：

- **macOS**：使用 Seatbelt 进行沙箱强制执行
- **Linux**：使用 **bubblewrap** 进行隔离
- **WSL2**：使用 bubblewrap，与 Linux 相同

不支持 WSL1，因为 bubblewrap 需要仅在 WSL2 中可用的内核功能。

这些操作系统级限制确保由 Claude Code 命令生成的所有子进程都继承相同的安全边界。

入门 Claude Code Docs

前置条件

在 **macOS** 上，沙箱使用内置的 Seatbelt 框架开箱即用。

在 **Linux 和 WSL2** 上，首先安装所需的包：

Ubuntu/Debian **Fedora**

```
sudo apt-get install bubblewrap socat
```

启用沙箱

您可以通过运行 `/sandbox` 命令来启用沙箱：

```
> /sandbox
```

这会打开一个菜单，您可以在其中选择沙箱模式。如果缺少所需的依赖项（例如 Linux 上的 `bubblewrap` 或 `socat` ），菜单会显示您平台的安装说明。

沙箱模式

Claude Code 提供两种沙箱模式：

自动允许模式：Bash 命令将尝试在沙箱内运行，并自动允许而无需权限。无法沙箱化的命令（例如需要访问非允许主机的网络访问的命令）会回退到常规权限流程。您配置的显式询问/拒绝规则始终受到尊重。

常规权限模式：所有 bash 命令都通过标准权限流程，即使是沙箱化的命令。这提供了更多控制，但需要更多批准。

在两种模式中，沙箱都强制执行相同的文件系统和网络限制。区别仅在于沙箱化命令是自动批准还是需要明确权限。

Claude Code Docs

自动允许模式独立于您的权限模式设置工作。即使您不在"接受编辑"模式中，当启用自动允许时，沙箱化的 bash 命令也会自动运行。这意味着在沙箱边界内修改文件的 bash 命令将执行而不提示，即使文件编辑工具通常需要批准。

配置沙箱

通过您的 `settings.json` 文件自定义沙箱行为。有关完整的配置参考，请参阅 **Settings**。

并非所有命令都与沙箱开箱即用兼容。一些可能帮助您充分利用沙箱的注意事项：

许多 CLI 工具需要访问某些主机。当您使用这些工具时，它们会请求访问某些主机的权限。授予权限将允许它们现在和将来访问这些主机，使它们能够在沙箱内安全执行。

`watchman` 与在沙箱中运行不兼容。如果您运行 `jest`，请考虑使用 `jest --no-watchman`

`docker` 与在沙箱中运行不兼容。考虑在 `excludedCommands` 中指定 `docker` 以强制其在沙箱外运行。

Claude Code 包含一个有意的逃生舱机制，允许命令在必要时在沙箱外运行。当命令由于沙箱限制（例如网络连接问题或不兼容的工具）而失败时，Claude 会被提示分析失败，并可能使用 `dangerouslyDisableSandbox` 参数重试命令。使用此参数的命令通过正常的 Claude Code 权限流程进行，需要用户权限才能执行。这允许 Claude Code 处理某些工具或网络操作无法在沙箱约束内运行的边界情况。

您可以通过在 **sandbox settings** 中设置 `"allowUnsandboxedCommands": false` 来禁用此逃生舱。禁用后，`dangerouslyDisableSandbox` 参数将被完全忽略，所有命令必须运行沙箱化或在 `excludedCommands` 中明确列出。

安全优势

防止提示注入

即使攻击者通过提示注入成功操纵 Claude Code 的行为，沙箱也确保您的系统保持安全：

文件系统保护：

无法修改关键配置文件，如 `~/.bashrc`

无法修改 `/bin/` 中的系统级文件

无法读取在您的 **Claude 权限设置** 中被拒绝的文件

网络保护：

Claude Code Docs

- 无法向攻击者控制的服务器泄露数据
- 无法从未授权的域下载恶意脚本
- 无法向未批准的服务进行意外的 API 调用
- 无法联系任何未明确允许的域

监控和控制：

- 所有在沙箱外的访问尝试都在操作系统级别被阻止
- 当边界被测试时，您会收到立即通知
- 您可以选择拒绝、允许一次或永久更新您的配置

减少攻击面

沙箱限制了以下可能造成的损害：

- **恶意依赖项**：具有有害代码的 NPM 包或其他依赖项
- **被破坏的脚本**：具有安全漏洞的构建脚本或工具
- **社会工程**：欺骗用户运行危险命令的攻击
- **提示注入**：欺骗 Claude 运行危险命令的攻击

透明操作

当 Claude Code 尝试访问沙箱外的网络资源时：

1. 操作在操作系统级别被阻止
2. 您会收到立即通知
3. 您可以选择：
    - 拒绝请求
    - 允许一次
    - 更新您的沙箱配置以永久允许它

安全限制

Claude Code Docs

网络沙箱限制：网络过滤系统通过限制进程允许连接的域来运行。它不会以其他方式检查通过代理的流量，用户负责确保他们仅在其策略中允许受信任的域。

> ⚠ 用户应该意识到允许广泛域名（如 `github.com` ）可能带来的潜在风险，这可能允许数据泄露。此外，在某些情况下，可能可以通过 **domain fronting** 绕过网络过滤。

通过 Unix Sockets 的权限提升： `allowUnixSockets` 配置可能会无意中授予对强大系统服务的访问权限，这可能导致沙箱绕过。例如，如果它用于允许访问 `/var/run/docker.sock` ，这将有效地通过利用 docker socket 授予对主机系统的访问权限。鼓励用户仔细考虑他们通过沙箱允许的任何 unix sockets。

文件系统权限提升：过于宽泛的文件系统写入权限可能导致权限提升攻击。允许写入包含 `$PATH` 中的可执行文件、系统配置目录或用户 shell 配置文件（ `.bashrc` 、 `.zshrc` ）的目录可能导致当其他用户或系统进程访问这些文件时在不同的安全上下文中执行代码。

Linux 沙箱强度：Linux 实现提供强大的文件系统和网络隔离，但包括一个 `enableWeakerNestedSandbox` 模式，使其能够在 Docker 环境中工作而无需特权命名空间。此选项大大削弱了安全性，应仅在其他隔离方式得到强制执行的情况下使用。

沙箱如何与权限相关

沙箱和 **permissions** 是互补的安全层，协同工作：

权限控制 Claude Code 可以使用哪些工具，在任何工具运行之前进行评估。它们适用于所有工具：Bash、Read、Edit、WebFetch、MCP 和其他工具。

沙箱提供操作系统级强制执行，限制 Bash 命令在文件系统和网络级别可以访问的内容。它仅适用于 Bash 命令及其子进程。

文件系统和网络限制通过权限规则配置，而不是沙箱设置：

使用 `Read` 和 `Edit` 拒绝规则来阻止对特定文件或目录的访问

使用 `WebFetch` 允许/拒绝规则来控制域名访问

使用沙箱 `allowedDomains` 来控制 Bash 命令可以到达的域名

此 **repository** 包含常见部署场景的启动设置配置，包括沙箱特定的示例。将这些用作起点，并根据您的需求进行调整。

高级用法

Claude Code Docs

自定义代理配置

对于需要高级网络安全的组织，您可以实施自定义代理来：

- 解密和检查 HTTPS 流量
- 应用自定义过滤规则
- 记录所有网络请求
- 与现有安全基础设施集成

```
{
  "sandbox": {
    "network": {
      "httpProxyPort": 8080,
      "socksProxyPort": 8081
    }
  }
}
```

与现有安全工具的集成

沙箱 bash 工具与以下工具配合使用：

- **权限规则**：与 结合以实现深度防御
- **开发容器**：与 一起使用以获得额外隔离
- **企业策略**：通过 强制执行沙箱配置

最佳实践

1. **从限制性开始**：从最小权限开始，根据需要扩展
2. **监控日志**：查看沙箱违规尝试以了解 Claude Code 的需求
3. **使用特定于环境的配置**：开发和生产环境的不同沙箱规则
4. **与权限结合**：将沙箱与 IAM 策略一起使用以实现全面安全
5. **测试配置**：验证您的沙箱设置不会阻止合法工作流程

开源 Claude Code Docs

沙箱运行时作为开源 npm 包提供，可用于您自己的代理项目。这使更广泛的 AI 代理社区能够构建更安全、更安全的自主系统。这也可以用于沙箱化您可能希望运行的其他程序。例如，要沙箱化 MCP 服务器，您可以运行：

```
npx @anthropic-ai/sandbox-runtime <command-to-sandbox>
```

有关实现细节和源代码，请访问 。

限制

性能开销：最小，但某些文件系统操作可能会稍微变慢

兼容性：某些需要特定系统访问模式的工具可能需要配置调整，或者甚至可能需要在沙箱外运行

平台支持：支持 macOS、Linux 和 WSL2。不支持 WSL1。计划支持原生 Windows。

另请参阅

- 全面的安全功能和最佳实践
  - 权限配置和访问控制
- 完整的配置参考
  - 命令行选项

此页面对您有帮助吗？ 是 否

权限 终端配置

Anthropic
Claude Code Docs
Careers
Economic Futures
Research
News
Trust center
Transparency

Availability
Status
Support center

Learn

Courses
MCP connectors
Customer stories
Engineering blog
Events
Powered by Claude
Service partners
Startups program

Terms and policies

Privacy policy
Disclosure policy
Usage policy
Commercial terms
Consumer terms